

MILITARY-GRADE AI INFRASTRUCTURE

Zero egress. Total auditability.

Sovereign and defence deployments cannot tolerate any uncontrolled path off the wire. systemprompt.io's air-gap topology runs the engine, Postgres, and a swappable on-prem inferencer on an internal: true Docker network — no gateway, no NAT, no host-routable interface. A single ingress socat container is the only demarcation. Un-allowed models are denied with a 403 before any upstream connection. Closure is asserted two independent ways: a network snapshot from a sidecar (ss/connttrack) and an app-level counter on the in-network responder.

AT A GLANCE

Sealed. Audited. Provable.

0

OUTBOUND PACKETS · ASSERTED

4

AUDIT-SPINE WRITES PER CALL

<300ms

GOVERNANCE P95 · SEALED RUN

100%

PROMPTS, TOOLS, MCP AUDITED

01 Core capabilities

>

SEAL

Sealed Docker network

internal: true on the app network. No gateway, no NAT, no host-routable path. The app container itself never joins a bridge.

Zero egress

>

DENY-FIRST

Pre-connect 403

Gateway allow-list enforced by PolicyResolver.resolve(). Un-listed models are denied before any upstream TCP connection — the deny path leaves no packet on the wire.

policy 403

>

AUDIT

4 writes per call

Every call writes governance_decisions → ai_requests INSERT → logs → ai_requests UPDATE. SIEM-ready JSON, in your Postgres, joined by trace_id.

In-DB audit

>

PROVE

Double-witnessed closure

(1) ss/connttrack from a monitor sidecar shows no external endpoint. (2) The in-network responder's /stats counter is unchanged on a denied-burst run.

Two proofs

02 Why sovereign deployments choose systemprompt.io

→

Belt-and-braces closure

Closure is asserted on two independent layers: network (Docker's internal: true attaches no gateway and no NAT to the network) and config (the gateway allow-list rejects un-listed models with a 403 before any upstream call). Neither alone has to be perfect.

→

Read-only profile dir, no boot-time writes

The profile mount is :ro. Pre-supply manifest_signing_secret_seed in secrets.json and the boot needs no writes at all. The secrets bootstrap probes writability (dir_is_writable) and degrades gracefully to an ephemeral seed when the dir is sealed.

→

Allow-list as version-controlled YAML

Model allow-list is services/ai/gateway-policies.yaml, ingested at every boot by run_gateway_policy_load in extensions/web/jobs/src/publish.rs with delete_orphans: true. No hand-seeded SQL; the security control lives next to the rest of the config.

TOPOLOGY	Two networks: airgap-internal (internal: true) and a tiny bridge for the ingress socat. The socat is the only container on both.
CLOSURE PROOFS	(1) ss -tunp + connttrack -L from a monitor sidecar; (2) in-network responder /stats unchanged on a denied-burst run. Both run by 01-egress-assert.sh .
IDENTITY	JWT-bound. RBAC + 4-stage governance pipeline (scope · secret-scan · blacklist · rate-limit) on every /v1/messages call.
DEPLOYMENT	Single Rust binary. :ro profile dir, no outbound calls required. Pre-supplied signing seed for stable identity across boots.
OBSERVABILITY	Audit-spine in your Postgres. Optional OpenTelemetry exporter; no required outbound telemetry path.
COMPLIANCE	SOC 2 (CC6.1, CC7.2), ISO 27001 (A.5.15, A.8.3, A.8.25), HIPAA §164.312, EU AI Act Art. 9/12/15, NIST AI RMF, OWASP Agentic Top 10. ITAR / IL-pattern sovereign deployments supported.

IN-NETWORK INFERENCE · SWAPPABLE

Demo: deterministic mock-inference (Anthropic Messages-compatible) Production: on-prem vLLM · Llama · Qwen · any /v1/messages

Endpoint resolution: \${ANTHROPIC_ENDPOINT} → in-network URL; provider stays enabled even without an upstream key.

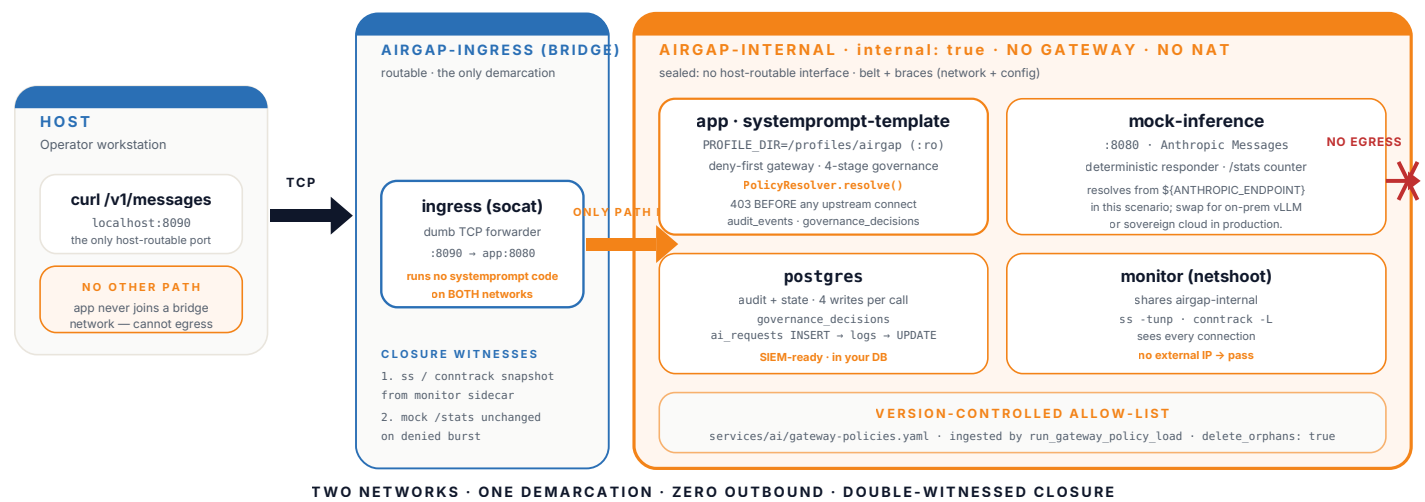
Evaluate the air-gap scenario free. **Sealed network, runnable on a laptop.**

`just airgap-up` brings up the sealed compose. `airgap-test` runs three assertion scripts: egress closure, sealed-network load, four-stage governance in isolation.

github.com/systempromptio/systemprompt-template

ed@systemprompt.io

03 Two networks, one demarcation: the sealed topology



ONE DEMARCATION The ingress socat is the only container on both networks; it forwards `:8090 → app:8080`. The app itself never joins a bridge — there is no host-routable interface for it to egress through, even if its config were misconfigured.

04 Sovereignty, on infrastructure you own

	Vendor SaaS	Cloud private	Air-gap + systemprompt.io
CLOSURE & RESIDENCY			
Outbound calls required	Yes · vendor cloud	Yes · cloud provider	None. <code>internal: true</code> · zero egress
Data residency	Vendor jurisdiction	Cloud region	Your datacenter. Your jurisdiction. ITAR / IL-pattern capable.
Deny-path proof	Vendor attestation	Cloud audit log	<code>/stats</code> counter unchanged · packet snapshot from sidecar
AUDIT & CONTROL			
Allow-list source-of-truth	Vendor console	Hand-seeded SQL / IaC	Version-controlled YAML · ingested every boot · <code>delete_orphans: true</code>
Audit ownership	Vendor-held	OpenTelemetry only	Your Postgres · 4 writes per call · joined by <code>trace_id</code>

05 Built for sovereign and defence deployments

FOR THE SOVEREIGNTY OFFICER
The proof is a packet snapshot.
No packet leaves the wall. Closure is asserted by a sidecar on the same sealed network — not by a vendor attestation, not by a cloud audit log. If the network ever opened, the next `01-egress-assert.sh` run would fail closed.
Outcome: Sovereignty you can re-prove on demand.

FOR THE AUDITOR
One SQL query, full lineage.
Every prompt → every policy decision → every tool call → every cost row, joined by `trace_id` to a JWT-verified identity. Run the query against your Postgres; the audit is structured evidence, not a policy document.
Outcome: Auditor asks, you run a SQL query.

Read the air-gap scenario walkthrough. Closure proofs, deny-first path, the four-stage governance pipeline.

Three runnable scripts: dual-layer egress assertion, sealed-network load + thresholds, four-stage governance in isolation. All artifacts under `results/`.

[demo/scenarios/airgap/architecture.md](#)